

PENSAMENTO COMPUTACIONAL



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Pensamento computacional: avaliação

Victor de Andrade Machado

OBJETIVOS DE APRENDIZAGEM

- > Identificar formas de avaliação da corretude de uma solução computacional.
- > Determinar métricas para avaliar a eficiência de soluções computacionais.
- > Apontar técnicas para avaliar usabilidade e simplicidade.

Introdução

Depois que uma solução foi projetada usando o pensamento computacional, é importante certificar-se de que ela é adequada para o propósito. A solução pode ser, por exemplo, um algoritmo para ser executado em um computador.

Deseja-se, efetivamente, começar a programar e, em passos paralelos e seguintes, realizar a avaliação de sua solução. Isso porque “Sem avaliação, nenhuma falha no algoritmo será detectada e o programa pode não resolver o problema corretamente, ou pode não o resolver da melhor maneira” (EVALUATING..., 2020).

Pense em avaliar o seu algoritmo como uma “depuração”. Avaliação é o processo que nos permite ter certeza de que a solução faz o trabalho para o qual foi projetada e pensar em como ela poderia ser melhorada. Ainda que você tenha chegado a uma solução, a princípio verídica, outros passos devem ser seguidos. A próxima etapa é avaliar a qualidade da solução como um todo, visando a ter certeza de que aquilo que foi produzido e/ou desenvolvido está dentro do esperado.

Neste capítulo, você conhecerá algumas técnicas relacionadas com a avaliação de soluções, observando o quão completas e eficientes elas podem ser. Além disso, conhecerá alguns conceitos relacionados à usabilidade, que mostra o quão o usuário do sistema sente ou não facilidade com as funcionalidades da solução, sua organização e seu *design*.

Analizando a solução proposta em corretude

Uma das perguntas mais importantes que devemos fazer acerca de uma solução é: ela realmente se propõe (e resolve) o problema originalmente proposto a ser resolvido? De forma geral, a pergunta que se faz é: esta solução está correta? Caso contrário, qualquer outra medida de qualidade (eficiência, usabilidade etc.) torna-se inválida, redundante. Ou seja, não importa o quão a sua solução é bem estruturada, rápida, fácil de mexer; se não resolver o seu problema, está incorreta, e você precisará realizar alterações.

Nesse sentido, alguns aspectos relacionados à qualidade devem ser considerados. Conforme Beecher (2017), deve-se questionar se a solução:

- **Possui corretude?** Essa questão enfatiza se o que foi desenvolvido realmente resolve o problema proposto.
- **É eficiente?** Não basta ser uma solução que resolva o problema, ela precisa trabalhar com a eficiência, ou seja, o uso otimizado dos recursos disponíveis.
- **É organizada?** Nesse contexto, verifica-se o fácil entendimento no desenvolvimento em geral, a sua eficácia e a possibilidade de outras pessoas continuarem ou realizarem a sua manutenção sem problemas.
- **É utilizável?** O foco aqui é o público-alvo do sistema, que deve ficar satisfeito com o sistema de forma geral, focando na intuição ao encontrar funcionalidades e sua aparência.

Se um algoritmo atender a esses quatro critérios, provavelmente funcionará bem. A falha na avaliação pode dificultar o funcionamento de um programa. Assim, a avaliação ajuda a garantir que o mínimo possível de dificuldades (ou nenhuma) seja enfrentado ao pôr em prática a solução.

Uma abordagem bastante utilizada é considerar que o programa (ou parte dele) está incorreto, até que não seja identificado erro e seja comprovado a sua correteza. O sentido dessa via de análise é que, na prática, quase nenhuma solução é considerada correta se não houver testes e depurações. Em termos de possibilidades, existem mais soluções incorretas em comparação com as soluções corretas. Uma abordagem semelhante é utilizada nos campos da ciência, medicina e direito, na qual se deve focar os que fazem afirmações positivas concretas, a fim de se proteger contra falso-positivos (BEECHER, 2017).

Na academia, falar em se provar alguma coisa geralmente está relacionado a uma prova matemática. Caso seja bem-sucedida, é estabelecida uma condição, de fato, incontestável. No entanto, essa abordagem é refutada por muitos profissionais quando uma abordagem mais complexa se relaciona, por exemplo, a sistemas críticos, com variações e análises constantes de possibilidades.

Na maioria das vezes, para mostrar que um programa está correto, é realizada, empiricamente, uma série de testes. No início de um processo de desenvolvimento, um problema é especificado de uma determinada forma. Essa especificação é refletida no plano de teste que reformula cada requisito como um teste. O teste executa cada aspecto da solução e verifica se o comportamento resultante atende à especificação.

Abordagem empírica

Uma **análise empírica** é aquela baseada na experimentação real e na observação dos resultados. Por exemplo, uma pesquisa está sendo conduzida para descobrir se ouvir música alegre enquanto trabalha pode promover a criatividade. Um experimento é conduzido por meio do uso de uma pesquisa de um *site* de música, com um conjunto de público exposto à música alegre e outro conjunto que não está ouvindo nenhuma música. Os sujeitos são, então, observados. Os resultados derivados de tal pesquisa fornecerão evidências empíricas de se a música alegre promove ou não a criatividade. No mundo dos algoritmos, isso indica que o algoritmo deve realmente ser traduzido para uma linguagem de programação e executado em um computador.

Um dos focos de uma abordagem de teste empírico é mostrar que a solução está incorreta. Essa consideração é mais fácil, pois apenas um teste com falha é suficiente para demonstrar a incorreção da solução. Em geral, é inviável testar todos os valores possíveis que podem ser inseridos em uma solução (e todas as combinações destes), portanto, cada teste aprovado adiciona mais uma peça para apoiar a evidência de uma solução correta.

Contudo, a redução do número de testes reprovados não fortalece a afirmação de que a solução está correta. A presença de um teste com uma única falha mostra que a solução está incorreta. De acordo com Beecher (2017):

- Sem aprovação e sem reprovação nos testes, não há evidência de correção. Isso porque é possível realizar um teste em um método ou classe que garanta que a solução está em pleno funcionamento. Contudo, se a funcionalidade não atende ao requisito inicial, a solução estará incorreta para o problema proposto.
- Alguns testes aprovados e nenhum teste falho significa que há alguma evidência de uma solução correta. Contudo, pode ser que, em alguma parte não testada da solução, seja encontrada uma falha não evidenciada por nenhum outro teste elaborado.
- Alguns testes aprovados e pelo menos um teste reprovado significa que há uma solução incorreta. Se há reprovação, então a solução deixa de executar corretamente alguma função, acarretando a incorretude como um todo.

A corretude mostra apenas que a solução resolve o problema. Não importa se ela é lenta, ineficiente, muito complicada ou outra pessoa simplesmente não gosta da solução, isso não afeta a corretude da solução para o problema definido.



Fique atento

Há uma razão pela qual a pesquisa empírica é um dos métodos mais amplamente utilizados: existem vantagens associadas a ela. A seguir, são apresentadas algumas delas.

- É utilizada para autenticar a pesquisa tradicional por meio de vários experimentos e observações.
 - Essa metodologia torna a pesquisa conduzida mais competente e autêntica.
 - A pesquisa empírica permite que o pesquisador entenda as mudanças dinâmicas que podem ocorrer e mude a sua estratégia de acordo com essas dinâmicas.
 - O nível de controle em tal pesquisa é alto para que o pesquisador possa observar múltiplas variáveis.
 - Desempenha um papel vital no aumento da validade interna, uma vez que se baseia em elementos palpáveis (reais), e não apenas em cogitações e simulações puras.
-

Eficiência das soluções

Um bom algoritmo está correto, mas um ótimo algoritmo é correto e eficiente. O algoritmo mais eficiente é aquele que leva a menor quantidade de tempo de execução e uso de memória possível, embora ainda produza uma resposta correta. É muito importante compreender completamente o problema a ser resolvido, destacando os pontos principais e os objetos que o compõem (ASCENCIO; CAMPOS, 2012).

Assim como uma prova de matemática formal se fundamenta em axiomas (as provas geométricas geralmente são baseadas nos axiomas da geometria euclidiana, ao passo que outras se fundamentam nos axiomas da teoria de conjuntos), uma prova formal de correção de um programa baseia-se nas especificações segundo as quais o programa foi projetado.

Cada algoritmo requer uma certa quantidade de recursos para fazer o seu trabalho. Algoritmos diferentes, ainda que resolvam o mesmo problema, podem ter um desempenho distinto em termos de eficiência. Esse desempenho geralmente é medido em termos de tempo e espaço.

- **Tempo:** duração do tempo de execução de um algoritmo, do início ao fim. A duração pode ser medida como o número de etapas realizadas durante a execução.
- **Espaço:** quantidade de armazenamento de memória exigida por um algoritmo para fazer o seu trabalho.

No entanto, a eficiência de um algoritmo pode realmente ser medida de forma confiável de uma forma que possa ser comparada com a de outros algoritmos? À primeira vista, pode parecer não ser o caso. Afinal, os algoritmos são executados em dados de entrada e têm um desempenho diferente dependendo do tamanho dos dados de entrada obtidos. Imagine um algoritmo que conta a quantidade de nomes de pessoas em uma lista de frequência, por exemplo. O seu tempo de processamento aumenta quanto mais números houver na lista, e aumentará ainda mais se for necessário pesquisar e comparar pessoas com o mesmo nome inicial.

Toscani e Veloso (2012) afirmam que, “[...] para um algoritmo rápido, qualquer melhora na velocidade de execução das operações básicas é sentida, e o conjunto de problemas que podem ser resolvidos por ele aumenta sensivelmente. Esse impacto é menor no caso de algoritmos menos eficientes”. A partir daí tem-se o conceito de **classe de complexidade**, com perfis ou categorias que informam como os algoritmos pertencentes a ela se comportarão

em geral. Assim, a medição da eficiência do algoritmo avalia a computação em geral, ou seja, fornece princípios gerais sobre a categoria que se pode aplicar a uma situação específica. A classe de complexidade não classifica todos os algoritmos possíveis e concebíveis em termos de eficiência, porém fornece um conjunto relativamente pequeno, em que cada algoritmo pode ser colocado, bem como leva em consideração que o tamanho dos dados de entrada pode variar.

Cada categoria descreve como os seus algoritmos funcionam no pior caso, ou seja, define a quantidade máxima de recursos que um algoritmo pode utilizar. Como o desempenho de um algoritmo depende dos dados de entrada, o uso de recursos é uma função do tamanho dos dados de entrada. O Quadro 1, a seguir algumas categorias de exemplo e sua eficiência em termos de tempo.

Quadro 1. Classes de complexidade usuais

Nome	Tempo de execução	Descrição
Constante	$O(1)$	Não leva, necessariamente, 1 unidade de tempo, mas sim um número fixo de unidades de tempo. Um exemplo é determinar se um número é positivo ou negativo.
Logarítmica	$O(\log N)$	À medida que o tamanho da entrada aumenta, o tempo de execução aumenta a uma taxa cada vez menor. As abordagens de dividir para conquistar geralmente funcionam assim.
Linear	$O(N)$	Esse é o melhor desempenho possível de qualquer algoritmo que deve passar por toda a entrada sequencialmente.
Quadrática	$O(N^2)$	O tempo de execução dos algoritmos nessa classe quadruplica quando o tamanho da entrada dobra.
Fatorial	$O(N!)$	Os algoritmos dessa classe são geralmente de força bruta e considerados muito lentos para uso prático.

Fonte: Adaptado de Beecher (2017).

Observe que o tempo de execução de cada categoria é expresso em uma notação chamada de 'Big-O' (O-grande), devido ao uso da letra maiúscula O. O termo dentro dos parênteses denota quanto tempo os algoritmos dessa classe geralmente requerem para processar a entrada. N representa o número de itens contidos na entrada.

Exemplo 1

Como visto anteriormente, pode-se categorizar o tempo de execução de um algoritmo com base em como o número de etapas aumenta conforme o tamanho da entrada. Sempre leva o mesmo tempo? Se sim, é um aumento constante, um tempo de execução muito rápido. É sempre necessário examinar todas as permutações possíveis da entrada? Se sim, é um aumento exponencial, um tempo de execução muito lento. A maioria dos tempos de execução estão entre os dois casos. Confira, a seguir, alguns exemplos relacionados aos graus de complexidade mais comuns.

Tempo constante

Quando um algoritmo é executado em tempo constante, ele sempre leva um número fixo de etapas, não importa o quão aumente o tamanho da entrada. Como exemplo, considere o acesso ao primeiro elemento de uma lista:

```
primeiroElemento ← nome [1]
```

Ainda que a lista aumente para um milhão de itens, essa operação sempre exigirá uma única etapa. Para uma lista de 10 nomes, o tempo será constante. Para uma lista de 1 milhão de nomes, o mesmo tempo será levado, afinal o problema é fixo, de modo que o processo será sempre o mesmo: obter o primeiro elemento.

Tempo logarítmico

Quando um algoritmo é executado em tempo logarítmico, ele aumenta proporcionalmente ao logaritmo do tamanho da entrada. O algoritmo de pesquisa binária, por exemplo, utiliza esse tipo de tempo.

```

1  PROCEDURE searchList(numbers, targetNumber) {
2      minIndex ← 1
3      maxIndex ← LENGTH(numbers)
4      REPEAT UNTIL (minIndex > maxIndex) {
5          middleIndex ← FLOOR((minIndex+maxIndex)/2)
6          IF (targetNumber = numbers[middleIndex]) {
7              RETURN middleIndex
8          } ELSE {
9              IF (targetNumber > numbers[middleIndex]) {
10                 minIndex ← middleIndex + 1
11             } ELSE {
12                 maxIndex ← middleIndex - 1
13             }
14         }
15     }
16     RETURN -1
17 }

```

Fonte: Fox ([2021], documento *on-line*).

O algoritmo usa um *loop* para examinar vários itens da lista, mas — o que é crucial — ele não examina todos os itens. Especificamente, ele examina $\log_2 n$ itens, onde n é o número de itens na lista. Na linha 1, é passada a lista (*numbers*) de números e o número a ser buscado (*targetNumber*). As linhas 2 e 3 recebem, respectivamente, o menor (no caso, 1) e o maior índice (tamanho da lista). Vale lembrar que, em muitas linguagens de programação, como Java e C, o índice é iniciado em zero, e o valor final passa a ser o tamanho da lista menos 01 unidade.

A linha 4 inicia uma repetição até que o tamanho do menor índice seja maior que o último índice. Ou seja, se o algoritmo for executado de forma que, após algumas atualizações, o menor índice ultrapasse o maior, o programa realizou as buscas sem encontrar o elemento requerido. A linha 5 realiza o cálculo para um índice central, que será a referência para o próximo passo da busca. Na linha 5, calcula-se a média dos índices para se saber o novo índice central. Se o valor for encontrado, esse índice é retornado (linha 6), do contrário, são feitas as seguintes avaliações:

- O número que se busca é maior que o número que está no índice central?
 - Se sim, a sua solução está mais à direita (considerando-se, visualmente, uma lista ordenada da esquerda para a direita), e o menor índice passa a ser o índice central, mostrado nas linhas 9 e 10.
 - Se não, o número está mais à esquerda, e o maior índice passa a ser o índice central, conforme o código nas linhas 11 e 12.

Após a execução, caso não seja encontrado o valor, o algoritmo retornará -1 (linha 16), indicando que não encontrou o valor solicitado.



Saiba mais

A pesquisa binária é um algoritmo eficiente para realizar a busca de um elemento em uma lista ordenada. Basicamente, ela funciona da seguinte forma:

- Uma lista é passada ao algoritmo (pode ser de números, nomes etc., contanto que esteja ordenada).
- Além da lista total, o valor a ser buscado também é passado como parâmetro.
- Define-se o menor e o maior índice (sendo o menor 1 ou 0, e o maior, o tamanho da lista ou o [tamanho da lista - 1]).
- Encontra-se o índice central, somando-se os dois extremos, e divide-se por 2. Se não for exato, arredonda-se (geralmente para cima).
- Atente-se ao exemplo: se o índice menor é 1 e o maior é 25, o índice central será 13 $([1 + 25]/2)$. Se o elemento do índice central (13) for 10, e o valor procurado for 20, então, como a lista é ordenada, o índice desse elemento, se houver, é maior que o do elemento central. Nesse caso, o índice 13 passa a ser o mínimo, e o 25, o máximo.
- Ao realizar um novo procedimento, o novo índice central será 19 $([13 + 25] / 2)$. Digamos, agora, que o elemento desse índice seja 27. Logo, o valor procurado é menor que o valor encontrado no índice central, portanto, o índice máximo passará a ser 19. Essas operações continuam até que o elemento seja encontrado ou que se finalize o processo de busca.

Observe que a quantidade de buscas não será igual à quantidade de elementos, devido ao funcionamento de divisão e à atualização do algoritmo, apontando para uma classe de tempo logarítmica.

Tempo linear

Quando um algoritmo cresce em tempo linear, o seu número de etapas aumenta em proporção direta ao tamanho da entrada. O algoritmo de busca linear apropriadamente denominado é executado em tempo linear (FERRAGINA; LUCCIO, 2018). Observe o pseudocódigo de busca linear a seguir:

```
1  PROCEDURE searchList (numbers, targetNumber) {  
2      index ← 1  
3      REPEAT UNTIL (index > LENGTH(numbers)) {  
4          IF (numbers[index] = targetNumber) {  
5              RETURN index  
6          }  
7          index ← index + 1  
8      }  
9      RETURN -1  
10 }
```

Dessa vez, o *loop* examina todos os itens da lista. Essa pesquisa exaustiva é necessária para buscar itens em uma lista não ordenada, pois não há como restringir onde um item específico pode estar. Esse algoritmo sempre exigirá, pelo menos, um número de passos igual ao número de itens. O *loop*, indicado na linha 3 do pseudocódigo, será realizado no pior caso até que todos os elementos sejam percorridos. Se o elemento em algum índice for encontrado (linha 4), a posição será retornada. Caso contrário, o valor -1 indicará que não foi encontrado o valor de busca (*targetNumber*).

Tempo quadrático

Quando um algoritmo cresce em tempo quadrático, os seus passos aumentam em proporção ao quadrado do tamanho de entrada. Vários algoritmos de ordenação de lista são executados em tempo quadrático, como ordenação por seleção (*selection sort*). Esse algoritmo começa no início da lista e, em seguida, continua encontrando o próximo valor menor na lista e trocando-o pelo valor atual.

Tempo exponencial

Quando um algoritmo cresce em tempo exponencial, o seu número de etapas aumenta mais rápido do que uma função polinomial do tamanho de entrada. Um algoritmo frequentemente requer tempo exponencial quando se deve olhar para todas as permutações de valores. Por exemplo, considere um algoritmo que gera todas as senhas numéricas possíveis para um determinado comprimento de senha. Para um comprimento de senha de 2, ele gera 100 senhas (00, 01, 02, ..., 99). Esse algoritmo requer, pelo menos, 102 passos, uma vez que existem 10 possibilidades para cada dígito (00–99) e ele deve tentar todas as possibilidades para cada um dos 2 dígitos.

Para qualquer comprimento de senha de n , o algoritmo requer $10n$ etapas. Esse tempo de execução aumenta incrivelmente rápido, pois cada dígito adicional requer 10 vezes o número de etapas. Pode-se, então, realizar um comparativo simples da complexidade variando o valor de n para ter uma visão de como a eficiência em cada caso irá se comportar, enfatizando a diferença entre cada um dos cálculos baseados no valor apontado.

O Quadro 2, a seguir, apresenta cálculos numéricos para dois valores N : 10 e 20. O intuito é mostrar o quão cada classe influencia a eficiência do algoritmo. Observe, no caso 1, que o logaritmo de 10 não se distancia muito do logaritmo de 20, com resultado considerado baixo em uma análise. No caso 2, na complexidade linear ($O(N)$), o valor calculado é igual ao valor de entrada. Na sequência, no terceiro caso, a complexidade $N\log(N)$ realiza um acréscimo no valor do logaritmo, multiplicado linearmente pelo valor de entrada. A partir do caso 4, a complexidade possui um maior crescimento, no qual, aumentando o valor de N , a complexidade aumenta bem mais nos casos seguintes, chegando a um valor de pior caso na complexidade fatorial (caso 6); Enquanto o fatorial de 10 é 3.628.800 (na casa dos 3 bilhões), o fatorial de 20 é cerca de 670 **bilhões** de vezes maior, provando o quão ineficiente pode ser um algoritmo nessa classe.

Quadro 2. Análise de memória de complexidade de algoritmos

Caso	Classe de complexidade	$N = 10$	$N = 20$
1	$O(\log(N))$	1	2,996
2	$O(N)$	10	20
3	$O(N\log(N))$	$10 \log (10) = 10$	$20 \log (20) = 59,9$
4	$O(N^2)$	100	400
5	$O(2N)$	1024	1048576
6	$O(N!)$	3628800	$2.43290200817e+18$

Fonte: Adaptado de Debnath (2018).

Assim, quanto maior a complexidade, o algoritmo tende a ser computacionalmente mais intensivo (o que não significa, necessariamente, que estará incorreto), o que, em geral, implica tempo de execução e capacidade de memória maiores.

Usabilidade e simplicidade de uma solução

Não menos importantes do que os outros critérios de qualidade, a usabilidade e a simplicidade ganham foco na avaliação de uma solução. Ou seja, aspectos humanos e seus hábitos particulares são levados em consideração, o que apontará se a solução realmente será utilizável e proporcionará para todos os seus usuários uma experiência positiva.

Beecher (2017) afirma que critérios como correção, eficiência e elegância descrevem uma solução em termos independentes, ou seja, cada um com suas características próprias. A usabilidade permite uma visão do quão bem, no sentido mínimo de facilidade e aprendizagem, as pessoas conseguem usar uma solução. Quanto menor for o esforço exigido, melhor será a usabilidade.

A usabilidade é um atributo de qualidade que avalia a facilidade de uso das interfaces de usuário. A palavra **usabilidade** também se refere a métodos para melhorar a facilidade de uso durante o processo de *design*. A usabilidade é definida por Beecher (2017) por meio de cinco componentes de qualidade:

1. **Aprendizagem:** os usuários realizam facilmente as tarefas, ainda que seja o seu primeiro contato com o *design* da solução?
2. **Eficiência:** com o contato realizado e as funções aprendidas, o usuário consegue trabalhar rapidamente e de forma fácil na interface?
3. **Memorização:** se os usuários passarem algum tempo sem utilizar a solução, quando retornarem, ainda lembrarão de como proceder em cada funcionalidade?
4. **Erros:** os usuários constantemente cometem erros, seja colocando uma letra no lugar de um número ou clicando em um local inadequado. O sistema consegue facilmente contornar esses erros e exibir o *feedback* positivo aos usuários?
5. **Satisfação:** o quão bem os usuários se sentem ao utilizar e visualizar o *design* de forma geral?

Outros atributos de qualidade também são considerados muito importantes. Entre eles, destaca-se a utilidade, que se refere a uma funcionalidade de *design*, enfatizando se ela realmente faz aquilo que os usuários precisam. A usabilidade e a utilidade devem andar juntas, considerando-se os seguintes argumentos: qual a importância de algo ser fácil se ele não realiza aquilo que você realmente precisa? Ou, então, considere que a solução é válida, no sentido de fazer o que se deseja, porém não se consegue realizar tais operações, pois a interface definida complica a busca das funções no sistema. Também não é uma proposta considerável. Para estudar a utilidade de um projeto, pode-se utilizar os mesmos métodos de pesquisa do usuário que melhoram a usabilidade.



Saiba mais

Confira, a seguir, as seguintes definições.

- **Utilidade:** os recursos necessários são fornecidos adequadamente?
- **Usabilidade:** esses mesmos recursos são agradáveis de serem utilizados?
- **Útil:** conceitos de usabilidade + utilidade.

Além desses critérios, outras abordagens muito utilizadas são a **elegância** e **simplicidade**. Vamos a um exemplo. Imagine que seja preciso realizar a soma de 10 números. Você cria uma lista com esses 10 números e realiza a soma conveniente da seguinte forma (considere `list_num` como o nome de sua lista):

```
soma = list_num[1] + list_num[2] + list_num[3] +
        list_num[4] + list_num[5] + list_num[6] + list_num[7] +
        list_num[8] + list_num[9] + list_num[10];
```

Realmente, o cálculo poderá estar correto. Mas essa solução está simples? Elegante? E se você utilizar estruturas conhecidas, facilitadoras de desenvolvimento? Observe o algoritmo a seguir:

```
i = 0;
soma = 0;
REPEAT UNTIL (i <= 9) {
    soma += list_num[i];
}
```

Qual a diferença? Elegância. No primeiro caso, você somou cada elemento individualmente, ao passo que, no segundo caso, criou uma estrutura para que o processo fosse automatizado. O cálculo é o mesmo? Sim, mas a forma como você trata cada solução é diferente.



Exemplo

Um professor pede para que a classe some todos os números de 01 a 100 e lhe entregue o resultado. O primeiro pensamento é sair somando cada elemento, um a um: $1 + 2 + 3 + 4 + \dots + 100$, obtendo-se o resultado esperado. Mas será que essa é a forma mais simples de se realizar o cálculo? Não vamos tratar em forma de programação, mas sim pensar computacionalmente, como se fosse fazer em papel. Se esse primeiro pensamento for colocado como resolução, não seria nada simples e elegante todo o trabalho para se somar cada elemento.

Imagine, agora, a seguinte situação: um aluno, como qualquer outro, avaliou o comportamento da soma. Se somar o primeiro e o último elementos, o resultado dará 101. Se somar o segundo e o penúltimo elementos, também dará 101. Ele observou, então, que $1 + 100 = 101$; $2 + 99 = 101$, etc. Seguindo essa sequência, a última soma seria $50 + 51 = 101$. Ou seja, serão feitas 50 somas cujo resultado é igual a 101. Logo, abstraindo esse comportamento, a solução encontrada foi: $50 \times 101 = 5.050$.

Qual a solução mais simples? Por meio de um pensamento computacional que analisou a sequência existente, a simplicidade no cálculo 50×101 se destaca na resolução do problema. Por isso, ainda que a solução encontrada possa ser eficiente e correta, a simplicidade e a elegância agregam um valor visual enriquecedor à solução, podendo, inclusive, influenciar em sua implementação sistemática posteriormente, reduzindo, por exemplo, linhas de código.

Referências

- ASCENCIO, A. F. G.; CAMPOS, E. A. V. *Fundamentos da programação de computadores: algoritmos, Pascal, C/C++ (Padrão ANSI) e Java*. 3. ed. São Paulo: Pearson, 2012. 569 p.
- BEECHER, K. *Computational thinking: a beginner's guide to problem-solving and programming*. Swindon: BCS Learning & Development, 2017. 288 p.
- DEBNATH, S. Analysis of Algorithms | Big-O analysis. *Geeks for Geeks*, Noida, 8 June 2018. Disponível em: <https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/>. Acesso em: 12 jan. 2021.
- EVALUATING solutions: Why do we need to evaluate our solutions? *BBC Bitesize*, London, 2021. Disponível em <https://www.bbc.co.uk/bitesize/guides/zssk87h/revision/2>. Acesso em: 12 jan. 2021.
- FERRAGINA, P.; LUCCIO, F. *Computational thinking: first algorithms, then code*. Cham: Springer Nature Switzerland, 2018. 174 p.
- FOX, P. Categorizing run time efficiency. *Khan Academy*, Mountain View, [2021]. Disponível em: <https://www.khanacademy.org/computing/ap-computer-science-principles/algorithms-101/evaluating-algorithms/a/comparing-run-time-efficiency>. Acesso em: 12 jan. 2021.
- TOSCANI, L. V.; VELOSO, P. A. S. *Complexidade de algoritmos*. 3. ed. Porto Alegre: Bookman, 2012. 280 p. (Série Livros Didáticos Informática UFRGS, 13).

Leituras recomendadas

- BROOKSHEAR, J. G. *Ciência da computação: uma visão abrangente*. 11. ed. Porto Alegre: Bookman, 2013. 576 p.
- DEMÉTRIO, V. H. C.; HRDLICKA, H. *Avaliação da qualidade de software percebida pelos usuários do sistema gerenciador da execução orçamentária, financeira e patrimonial do Governo Federal - SIAFI*. Rio Tinto: Universidade Federal da Paraíba, 2017. 16 p. Disponível em: <https://repositorio.ufpb.br/jspui/handle/123456789/4451>. Acesso em: 12 jan. 2021.
- HØG, E. et al. The Tycho-2 catalogue of the 2.5 million brightest stars. *Astronomy and Astrophysics*, [S. l.], v. 355, p. L27–L30, 2000. Disponível em: <http://adsabs.harvard.edu/full/2000A&A...355L..27H>. Acesso em: 12 jan. 2021.
- RILEY, D. D.; HUNT, K. A. *Computational thinking for the modern problem solver*. Boca Raton: CRC Press, 2014. 406 p. (Chapman & Hall/CRC Textbooks in Computing).



Fique atento

Os links para sites da web fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais links.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS